DB

Databricks Certified Generative AI Associate

New & High-Frequency Topics — Deep-Dive Notes

Study Guide with Tables · Code · Exam Traps · Cheat Sheet

#	Topic
1	Agent Bricks ★ HIGH FREQUENCY
2	MCP — Model Context Protocol
3	AI Gateway (Unity AI Gateway) ★ HIGH FREQUENCY
4	MLflow 3 Tracing ★ HIGH FREQUENCY
5	Prompt Registry & Prompt Versioning
6	Mosaic AI Agent Evaluation ★ HIGH FREQUENCY
7	Custom Scorers (& Custom Judges)
8	Lakehouse Monitoring

Companion to your Domain 1–6 study guides & the 200-question practice sets. Verified against Databricks documentation, 2026.

1. Agent Bricks

★ HIGH FREQUENCY

Agent Bricks is Databricks' governed, largely no-code platform for building, deploying, and governing enterprise AI agents that operate on your own business data end to end. You give a high-level description of the task and connect your data; Agent Bricks auto-generates domain-specific synthetic data and task-aware benchmarks (via Mosaic AI Research techniques) and then auto-optimizes for the cost/quality trade-off you choose.

The exam tests which Brick fits which use case. Memorize the table below — the wrong-answer options almost always swap one Brick for another.

Agent Brick	Use it for	Tell-tale signal in the question
Knowledge Assistant (GA)	High-quality chatbot / Q&A grounded in your documents, with citations	"answer questions", "chatbot", "cite sources", "over our docs"
Information Extraction / Document Intelligence (GA)	Turn unstructured docs (PDFs, emails, contracts) into structured JSON / tables	"extract fields", "structured JSON", "names/dates/amounts", "at scale"
Multi-Agent Supervisor (GA)	Orchestrate multiple agents + tools (Genie, KAs, UC functions, MCP) into one workflow	"coordinate", "route subtasks", "intent + retrieval + compliance", parallel/specialized agents
Genie / AI-BI Genie	Natural-language analytics over governed tables (text-to-SQL style)	"ask questions about our tables/warehouse", BI-style numeric answers
Custom LLM	Specialize an LLM for a custom text task (summarize, classify, transform)	"custom text transformation", "specialize a model" without RAG/extraction

Supervisor Agent — key facts

- Orchestrates subagents (called 'tools'): Knowledge Assistants, Genie Spaces, Unity Catalog functions, other agent endpoints.
- Hard limit: no more than 20 agents in a single supervisor system.
- AI Search index subagents support only Delta Sync indexes.
- Supports long-running task mode: breaks complex tasks into multiple request/response cycles (task continuation) to avoid timeouts.
- Testable in AI Playground before going live; you can attach response guidelines in natural language to steer behavior.

Governance & integration (why it's 'enterprise')

- Agents connect directly to the lakehouse as the governed source of truth; existing Unity Catalog access controls are preserved.
- Natively supports MCP (Model Context Protocol) for secure tool/API/SaaS access.
- Persistent agent memory is provided via Lakebase, stored in your lakehouse.
- Built-in evaluation: auto-built benchmarks + LLM judges; choose cost-optimized or quality-optimized.

Key points

- ✓ Agent Bricks = managed, governed agent platform (low-code) — fastest path to production.
- ✓ Knowledge Assistant = Q&A/chatbot with citations. Information Extraction = unstructured → structured JSON.
- ✓ Supervisor = multi-agent orchestration (max 20 subagents).
- ✓ Custom agents (code-your-own with LangChain/LangGraph/LlamaIndex) trade speed for maximum flexibility.

EXAM TRAP — Classic trap: a PDF-to-JSON task offered with 'Knowledge Assistant' as a tempting wrong answer. Extraction → Information Extraction Brick; Q&A → Knowledge Assistant. Another trap: choosing Agent Bricks for its 'flexibility' — flexibility is the advantage of CUSTOM agents; the Brick's advantage is speed-to-production + built-in governance/eval.

2. MCP — Model Context Protocol

MCP (Model Context Protocol) is an open standard for connecting LLMs/agents to external tools, data sources, and services through a single, uniform interface. Instead of bespoke glue code per integration, an agent can discover and call tools/resources in a consistent way — which is why it is described as the emerging standard for tool integration.

How it shows up on Databricks

- Agent Bricks natively supports MCP, giving agents secure access to APIs, databases, and SaaS apps.
- Managed OAuth MCP Connectors let you connect external services (e.g., GitHub, Atlassian/Glean, Google Drive, SharePoint) as GOVERNED tools without registering your own OAuth app or managing credentials.
- AI Gateway extends governance to MCP-connected tools — identity, permissions, observability, and guardrails apply to tool calls too.

Why it matters for design questions

- MCP standardizes the tool layer so agents are portable and not locked to one integration style.
- Security/governance is the selling point on the exam: MCP tools on Databricks inherit Unity Catalog / AI Gateway controls (identity-first, credentials don't leak).
- Pair MCP (the connection standard) with the Tool-Use agentic pattern (the behavior of deciding when to call a tool).

Key points

✓ MCP = open, uniform standard for agent↔tool/data connectivity.

✓ On Databricks: native in Agent Bricks; Managed OAuth MCP Connectors; governed by AI Gateway + Unity Catalog.

✓ Think 'standardized, secure, interoperable tool access' when you see MCP in an answer.

EXAM TRAP — Don't confuse MCP (the connection/integration standard) with function/tool calling (the model capability to emit a structured call). MCP is the plumbing; tool-calling is the model behavior. Both can be true at once.

3. AI Gateway (Unity AI Gateway) ★ HIGH FREQUENCY

AI Gateway is a unified governance/management layer that sits in front of model serving endpoints (and now MCP-connected tools). It extends Unity Catalog governance from assets (tables, models, functions) to the RUNTIME behavior of AI systems: model requests, agent activity, tool calls, traces, guardrail enforcement, and inference logs.

The five capability areas (memorize these)

Capability	What it does
Rate limiting & traffic	Per-user/endpoint rate limits, load balancing, traffic controls across models/agents/endpoints.
Usage tracking & cost	Track token consumption, attribute cost by user/team/app/provider, enforce budgets, auto-stop spend at thresholds.
Inference tables (logging)	Auto-capture prompts, responses, tokens, latency, traces, user identity, guardrail outcomes to a UC Delta table.
Guardrails	Detect/block PII exposure, unsafe content, prompt injection, data exfiltration, hallucinations (configurable).
Routing / fallback	Auto-route to backup models during outage or quality degradation; external-model & multi-provider routing.

Inference tables vs Usage tracking (high-yield distinction)

	Inference tables	Usage / system tables
Captures	Full request + response payloads, latency, status, traces, identity, guardrail outcome	Aggregate consumption: tokens, request counts, cost attribution
Stored as	UC-governed Delta table (default name <catalog>.<schema>.<endpoint>_payload)	system.serving.endpoint_usage / served_entities
Used for	Debugging, evaluation, drift monitoring, building training/fine-tune corpus	Cost analysis, budgets, chargeback, usage SQL/alerts

```
-- Join usage + entity system tables to monitor an endpoint's consumption
SELECT user_id, endpoint_name,
SUM(num_tokens) AS total_tokens,
COUNT(*) AS total_requests
FROM system.serving.endpoint_usage AS eu
JOIN system.serving.served_entities AS se
ON eu.served_entity_id = se.served_entity_id
WHERE endpoint_name = '<your_endpoint>'
GROUP BY user_id, endpoint_name
ORDER BY total_tokens DESC;
```

Key points

- ✓ AI Gateway = runtime governance in front of endpoints: rate limits, usage/cost, inference tables, guardrails, routing/fallback.
- ✓ Inference tables = detailed payload logging (content). Usage/system tables = consumption accounting (cost).
- ✓ Enable in the Serving UI → 'AI Gateway' section. Config changes apply in ~20–40s; rate-limit changes up to ~60s.

EXAM TRAP — Two 2026 traps: (1) AI Gateway does NOT rate-limit ai_query() batch inference — it only provides usage TRACKING for it; control batch cost via endpoint permissions or system-table alerts. (2) Legacy inference tables are retiring (no support after Apr 30 2026) — the recommended choice is AI Gateway-enabled inference tables. Max logged payload size is 1 MiB; logs may be skipped on 4xx/5xx errors.

4. MLflow 3 Tracing ★ HIGH FREQUENCY

MLflow 3 for GenAI unifies tracking, evaluation, and observability across the dev-to-prod lifecycle. MLflow Tracing is the observability foundation: it records the step-by-step execution of a GenAI app/agent as structured spans and logs the trace data that evaluation and monitoring build on.

What a trace captures

- Nested spans for each step: query embedding, retrieval (which chunks + scores), the assembled prompt, each LLM call (with input/output token counts), tool invocations, and parsing.
- Latency and token usage per step — so you can find cost/latency bottlenecks.
- Inputs and outputs at each span — so you can see whether retrieval returned the right context or a tool errored.
- Real-time trace logging in development, testing, AND production (same traces feed evaluation/monitoring).

How tracing is enabled

- Automatic: mlflow.openai.autolog() (and 20+ integrations — Anthropic, LangChain/LangGraph, LlamaIndex, etc.).
- Manual: decorate functions with @mlflow.trace to control how spans are organized.
- Traces can be stored in AI Gateway inference tables for deployed agents — linking serving logs to observability.

```
import mlflow
from databricks_openai import DatabricksOpenAI

mlflow.openai.autolog() # auto-trace every LLM call
client = DatabricksOpenAI()

@mlflow.trace # manual span around your own logic
def answer(question: str):
# retrieval + prompt + client.chat... -> nested spans appear in the trace
...
```

Key points

✓ Tracing = structured, span-level record of an app's execution (inputs/outputs, latency, tokens, tool calls).

✓ It is the FOUNDATION for evaluation and production monitoring — same traces, dev and prod.

✓ Enable via autolog() or @mlflow.trace. Debug: 'did retrieval get the right context?' and 'where's the latency?'

EXAM TRAP — Tracing is observability (what happened), not evaluation (how good it was) — but evaluation consumes traces. If a question asks how to DEBUG which step caused a bad RAG answer, the answer is Tracing; if it asks how to SCORE quality, that's mlflow.evaluate + scorers.

5. Prompt Registry & Prompt Versioning

The MLflow Prompt Registry stores prompts as named, versioned artifacts with lifecycle management, governed by Unity Catalog. It solves untracked prompt drift: prompts become first-class, version-controlled assets you can compare, roll back, reuse across apps, and promote safely — just like models. The 2026 exam explicitly tests prompt versioning/lifecycle.

What you get

- Versioning: every prompt edit is a new version; compare versions and track quality improvements over iterations.
- Aliases/lifecycle: reference a stable alias (e.g., production) in code while swapping the underlying version — enables safe promotion, rollback, and A/B testing without code changes.
- Lineage & governance: Unity Catalog provides consistent governance across prompts, apps, and traces.
- Tight integration with evaluation: evaluate each prompt version on the SAME dataset to pick the best one.

```
# Compare two prompt versions on one fixed eval dataset
from mlflow.genai.scorers import Correctness

for version in [1, 2]:
with mlflow.start_run(run_name=f"summary_v{version}_eval"):
mlflow.log_param("prompt_version", version)
mlflow.genai.evaluate(
predict_fn = create_summary_function(PROMPT_NAME, version),
data = eval_dataset, # same data for fair comparison
scorers = [Correctness(), sentence_count_judge],
)
```

Key points

✓ Prompt Registry = version control + lifecycle (aliases) + UC governance for prompts.

✓ Reference an alias in code; change the version behind it to promote/rollback safely.

✓ Best practice: evaluate all prompt versions on the same dataset; track versions, results, and decisions.

EXAM TRAP — 'Copy-paste the prompt into each notebook' and 'bake the prompt into model weights' are always wrong. Prompts live in the registry, are versioned, and are referenced by alias.

6. Mosaic AI Agent Evaluation ★ HIGH FREQUENCY

Agent Evaluation measures the QUALITY of agent/RAG outputs — distinct from the Agent Framework (which BUILDS/deploys agents). In MLflow 3, the Agent Evaluation SDK is integrated into Databricks-managed MLflow, so the same judges/scorers are used in development AND production monitoring, ensuring consistent evaluation across the lifecycle.

How it works

- Builds on MLflow Tracing: traces are evaluated with built-in or custom LLM judges and scorers.
- Run via mlflow.genai.evaluate(data=..., predict_fn=agent, scorers=[...]).
- Evaluation Datasets give versioning, lineage, and UC integration; you can convert traces → eval records, or use a small list of dicts for quick prototyping (<100 examples).
- Review App: domain experts (SMEs) give feedback on traces, producing labeled evaluation data for the next iteration. Feedback attaches to traces with metadata (user, timestamp, revisions).

CLEARS — the standardized agent-quality framework (MLflow)

Letter	Dimension
C	Correctness
L	Latency
E	Execution (did tool use / steps run properly)
A	Adherence (to instructions/guidelines)
R	Relevance
S	Safety

Common built-in scorers

- Correctness — does the answer match expected facts.
- Safety — is the output safe/policy-compliant.
- RAG-specific judges: faithfulness/groundedness, answer relevancy, context precision/recall (see Eval-metrics notes).

Key points

✓ Agent Evaluation = measure quality (judges + scorers + SME review). Agent Framework = build/deploy.

✓ Integrated into managed MLflow 3; same scorers dev → prod.

✓ Review App collects SME feedback → builds golden datasets → calibrates judges.

✓ Remember CLEARS: Correctness, Latency, Execution, Adherence, Relevance, Safety.

EXAM TRAP — Don't pick 'Agent Framework' when the question is about SCORING/measuring quality, or 'Agent Evaluation' when it's about BUILDING the agent. Also: the new GenAI eval suite is available in MANAGED MLflow on Databricks.

7. Custom Scorers (& Custom Judges)

When built-in metrics don't capture domain-specific quality, you define a CUSTOM scorer: a user-defined evaluation function (heuristic/code-based OR an LLM judge) that returns a metric per example and plugs into the same mlflow.genai.evaluate harness used in dev and prod.

Two flavors

Type	How	Best for
Code-based scorer	Plain Python check (regex/schema/rule) returning a score/label	Deterministic business rules: valid JSON, contains a valid ticket/policy ID, length limits
LLM judge (make_judge)	Natural-language rubric scored by an LLM	Subjective/semantic checks: tone, helpfulness, 'exactly 2 sentences', adherence to guidelines

```
from typing import Literal
from mlflow.genai import make_judge
from mlflow.genai.scorers import Correctness

# Custom LLM judge with a rubric
sentence_count_judge = make_judge(
name="sentence_count_compliance",
instructions="""Evaluate if this summary follows the 2-sentence
requirement. Summary: {{ outputs }}
Count the sentences and decide if it has exactly 2.""",
feedback_value_type=Literal["correct", "incorrect"],
)

results = mlflow.genai.evaluate(
data=eval_dataset,
predict_fn=agent,
scorers=[Correctness(), sentence_count_judge], # built-in + custom
)
```

Key points

- ✓ Custom scorer = your own metric (code rule or LLM judge via make_judge) returning a score per example.
- ✓ Use when built-ins miss domain/business correctness (schema-valid JSON, must cite a valid ID, exact format).
- ✓ Same scorers run in development and production monitoring — consistency by design.

EXAM TRAP — BLEU/ROUGE/perplexity can't check business rules like 'must include a valid ticket ID' — that needs a custom scorer. 'Eyeball a few examples' never scales and is always the wrong answer for evaluating at scale.

8. Lakehouse Monitoring

Lakehouse Monitoring (built on Unity Catalog) maintains profile and drift metrics on your data and AI assets, auto-generates quality dashboards, supports proactive alerts, and aids root-cause analysis by correlating data-quality issues across the lineage graph. For GenAI you typically monitor the (unpacked) inference table of a served app.

Three monitor (profile) types — know which fits

Monitor type	Use when	Example
InferenceLog	Monitoring model/agent serving requests over time with model-quality + drift metrics	RAG endpoint payload table: track quality, drift, error rate
TimeSeries	A table has a timestamp column and you want metric trends over time (not a serving log)	Daily aggregated metrics on a Delta table
Snapshot	Evaluate the full current state of a table (no time dimension)	One-off data-quality profile of a dataset

Typical GenAI monitoring workflow

- Enable AI Gateway inference tables on the endpoint (captures requests/responses/traces).
- Schedule a job to unpack the JSON payloads per the endpoint schema (starter notebooks exist).
- (Optional) Join with ground-truth labels to compute model-quality metrics.
- Create a monitor over the resulting Delta table; refresh metrics; build dashboards + alerts.

Production metrics to watch

- Operational: QPS (throughput), latency, error rate, cost / token usage.
- Quality: faithfulness, answer relevancy, correctness (via scorers) on sampled traffic; user feedback/thumbs.
- Drift: shifts in input distribution (data drift) or input→output expectations (concept drift) → alert + refresh.

Key points

✓ Lakehouse Monitoring = UC-based profiling/drift + dashboards + alerts over your data/AI tables.

✓ Three monitors: InferenceLog (serving logs), TimeSeries (timestamped metrics), Snapshot (current state).

✓ GenAI flow: AI Gateway inference table → unpack → (join labels) → create monitor → dashboards/alerts.

EXAM TRAP — Map the monitor to the data shape: serving request logs → InferenceLog; a timestamped metrics table → TimeSeries; a static current-state table → Snapshot. 'Schema monitor' / 'Judge monitor' are not real types.

Tricky Exam Questions & Answers

Q1. You must turn 400,000 unstructured contracts into queryable structured fields with no custom pipeline. Which Agent Brick?

A: Information Extraction / Document Intelligence Brick — it converts unstructured docs to structured data at scale. Knowledge Assistant is for Q&A, not extraction.

Q2. Your agent needs governed, credential-safe access to GitHub and SharePoint as tools. What do you use?

A: Managed OAuth MCP Connectors (MCP) — they connect external services as governed tools without you managing OAuth/credentials, and AI Gateway governs the tool calls.

Q3. Finance wants per-team token-cost attribution and monthly budgets across several LLM endpoints. Which feature?

A: AI Gateway usage tracking (system.serving.endpoint_usage + served_entities) with budgets/auto-stop. Inference tables log payloads but the COST/consumption view comes from usage/system tables.

Q4. A RAG answer is wrong and you must find whether retrieval or generation caused it. What do you inspect?

A: MLflow Tracing — the trace's retrieval span shows which chunks/scores were returned and the LLM span shows the prompt/output, pinpointing the failing step.

Q5. You want to safely roll out prompt v2 and revert instantly if quality drops, without changing app code. How?

A: Reference a Prompt Registry ALIAS (e.g., production) in code; point it at v2, and roll back by repointing to v1. Combine with mlflow.genai.evaluate on a fixed dataset to gate the change.

Q6. Answers must always include a valid policy number. How do you measure compliance across 5,000 eval rows?

A: A custom scorer — a code-based check (regex/validation) or a make_judge rubric — run inside mlflow.genai.evaluate. BLEU/ROUGE/perplexity cannot check this rule.

Q7. Is the Agent Framework or Agent Evaluation responsible for measuring correctness, latency, and safety of an agent?

A: Agent Evaluation (integrated into managed MLflow 3). The CLEARS framework covers Correctness, Latency, Execution, Adherence, Relevance, Safety. The Agent Framework builds/deploys the agent.

Q8. You enabled monitoring on a served RAG app's payload table to track drift and quality over time. Which monitor type?

A: InferenceLog — it's purpose-built for model/agent serving request logs with quality + drift metrics. TimeSeries is for generic timestamped metric tables; Snapshot is current-state only.

Q9. True/False: AI Gateway rate limiting will throttle ai_query() batch inference jobs.

A: FALSE (2026). AI Gateway provides usage TRACKING for ai_query but does not enforce rate limits on batch inference. Control batch cost via endpoint permissions or system-table alerts/budgets.

Q10. Which Brick orchestrates a workflow across a Genie Space, a Knowledge Assistant, and a UC function — and what's its hard limit?

A: Multi-Agent Supervisor. Hard limit: at most 20 agents (subagents/tools) in one supervisor system; AI Search subagents must use Delta Sync indexes.

Last-Minute Cheat Sheet

Concept	One-liner
Agent Bricks	Managed, governed, low-code agent platform; auto-builds benchmarks+judges; cost/quality optimize.
• Knowledge Assistant	Doc Q&A chatbot with citations.
• Information Extraction	Unstructured → structured JSON at scale.
• Supervisor (≤20 agents)	Orchestrate agents/tools (Genie, KA, UC fn, MCP).
• Custom / Code-your-own	Max flexibility via LangChain/LangGraph/LlamaIndex.
MCP	Open standard for agent↔tool/data; Managed OAuth connectors; governed by AI Gateway+UC.
AI Gateway	Runtime governance: rate limits, usage/cost, inference tables, guardrails, routing/fallback.
• Inference tables	Payload logging (content) → UC Delta table (...endpoint_payload).
• Usage/system tables	system.serving.endpoint_usage + served_entities → cost/consumption.
• ai_query note	Usage-tracked but NOT rate-limited by gateway (2026).
MLflow 3 Tracing	Span-level observability (inputs/outputs, latency, tokens, tool calls); dev+prod.
• Enable	mlflow.openai.autolog() or @mlflow.trace.
Prompt Registry	Versioned, UC-governed prompts; aliases for promote/rollback; eval per version.
Agent Evaluation	Quality measurement (judges+scorers+Review App); CLEARS; managed MLflow 3.
Custom scorers	Code rule OR make_judge LLM rubric in mlflow.genai.evaluate; for business correctness.
Lakehouse Monitoring	UC profiling/drift + dashboards + alerts. Monitors: InferenceLog / TimeSeries / Snapshot.

Tip: these eight topics are the newest material and disproportionately likely to appear. If you can reproduce this cheat sheet from memory, you're in good shape.